

CSS

IS231: Web Technology

Cascading Style Sheets (CSS)

By: Neamat El Tazi

References

Figures & Slide References

Most content, figures, and examples are adapted from:

• **W3Schools.com** — The world's largest web developer resource

 <https://www.w3schools.com/css/>

HTML Style Attribute

Applying CSS inline with the style attribute — font, color, and size

Example 1 — Font Family & Background

```
<h1 style="font-family:verdana;">This is a heading</h1>  
<p style="font-family:courier;">This is a paragraph.</p>
```

Example 1 - Output

This is a heading

This is a paragraph.

Example 2 — Font Size & Text Alignment

```
<!DOCTYPE html>  
<html>  
  <body>  
  
    <p>I am normal</p>  
    <p style="color:red;">I am red</p>  
    <p style="color:blue;">I am blue</p>  
    <p style="font-size:50px;">I am big</p>  
  
  </body>  
</html>
```

Example 2 — Output

I am Red

I am Blue

I am Big

Cascading Style Sheets (CSS)

The language that styles the web — colors, layouts, fonts & more

What is CSS?

The language used to style HTML documents — controlling colors, fonts, and layouts.

How does CSS work?

CSS rules select HTML elements and define how they are displayed on screen or paper.

Why use CSS?

One stylesheet controls multiple web pages, saving time and keeping designs consistent.

```
/* CSS: selector { property: value; } */  
h1 { color: crimson; font-size: 2em; background: #f0f0f0; }  
p { font-family: Verdana, sans-serif; margin: 20px; line-height: 1.6;  
}
```

Why do we need CSS?

Messy HTML Styles

Style attributes scattered in HTML create duplication and maintenance nightmares.

Separate Concerns

CSS keeps style separate from content — add it once, remove all in-HTML duplicates.

Save Work

One CSS file controls layout and design for all your web pages at once.

External Files

Stylesheets are stored in separate .css files, easy to share and reuse across projects.

CSS Solved a Big Problem

⚠ **The Problem**

HTML was NEVER intended to contain formatting tags.

HTML was created to describe content:

```
<h1>This is a heading</h1>
```

```
<p>This is a paragraph.</p>
```

Adding `<font>` and color to every HTML 3.2 page became a long, expensive nightmare for developers.

✅ **The Solution: CSS**

W3C created CSS to solve the formatting chaos.

CSS removed style from HTML — keeping structure and presentation separate.

One stylesheet → consistent look across all pages.

CSS Saves a Lot of Work

Style definitions live in external .css files

All your CSS rules are written once in a separate file — not mixed into every HTML page.

Change one file — update the entire website

With an external stylesheet, modifying one .css file instantly updates every HTML page that links to it.

```
<link rel="stylesheet" href="mystyle.css">    /* in <head> of every HTML
page */
/* mystyle.css */
body { font-family: Verdana; background: #f8f8f8; }
```

CSS Demo



Run `css_demo.html` to see how different "Styles" elements change the whole page appearance.

Switch Stylesheets

See the entire page transform by linking a different .css file.

Colors & Fonts

Watch how background-color, font-family, and color properties apply instantly.

Layout Changes

Observe how margin, padding, and layout shift across different style sheets.

CSS Syntax

```
/* selector { property: value; } */  
h1 { color: blue; font-size: 12px; }
```

Selector

Points to the HTML element to style

Declaration Block

Wrapped in { };
multiple declarations allowed

Property : Value

Property + value pair,
separated by colon

How To Add CSS

When a browser reads a style sheet, it formats the HTML document according to the information in the sheet.



External CSS

Separate .css file linked with `<link>`
— best for multi-page sites.

`mystyle.css`



Internal CSS

`<style>` block inside `<head>` — good for single-page unique styles.

`<style>` in `<head>`



Inline CSS

style attribute on an element — for one-off, single-element styling.

`style="..."`

External CSS

With an external style sheet, you can change the look of an entire website by changing just one file!

Each HTML page must include a reference to the external style sheet inside the `<link>` element, in the `<head>` section.

```
<!DOCTYPE html>
<head>
  <link rel="stylesheet" href="mystyle.css">
</head>
```

External CSS

External .css files are written in any text editor and saved with a .css extension.

The .css file should NOT contain any HTML tags — only CSS rules.

```
/* mystyle.css — no HTML tags here! */  
body { background-color: lightblue; }  
h1 { color: navy; margin-left: 20px; }
```

Internal CSS

Internal CSS

Used when one single HTML page has a unique style — keeps style isolated to just that page.

Where to define it

Inside a `<style>` element, in the `<head>` section of the HTML file.

```
<!DOCTYPE html>
<html><head>
<style>
  body { background-color: linen; }
  h1   { color: maroon; margin-left: 40px; }
</style>
</head><body>
<h1>This is a heading</h1>
<p>This is a paragraph.</p>
</body></html>
```

Inline CSS

Inline CSS

Used to apply a unique style to a single HTML element — not reusable across the page.

How to use

Add the style attribute directly to the element. Can contain any CSS property.

```
<!DOCTYPE html>
<html><body>

<h1 style="color:blue;text-align:center;">This is a heading</h1>
<p style="color:red;">A red paragraph.</p>

</body></html>
```

Multiple Style Sheets

Multiple Stylesheets

If the same selector appears in different style sheets, the value from the LAST read stylesheet wins.

Override Example

mystyle.css: h1 { color: blue } but mystyle2.css (loaded after): h1 { color: red } — red wins.

```
/* mystyle.css - linked first */
h1 { color: blue; }
p { color: red; }

/* mystyle2.css - linked after (overrides!) */
h1 { color: red; }
```

Cascading Order

1 — Inline Style (highest)

style attribute on an HTML element always wins.

2 — External & Internal CSS

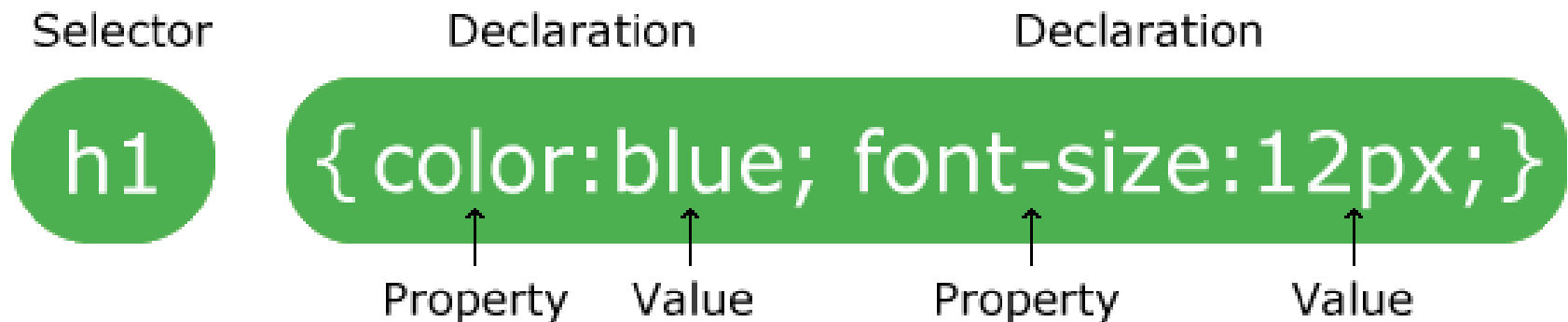
In the <head>. External or internal — whichever comes last is applied.

3 — Browser Defaults (lowest)

Built-in default styles applied when no other rule matches.

```
<p style="color:pink">Inline wins!</p> /* Priority 1 */  
<style>p{color:blue}</style>           /* Priority 2 */  
/* browser default */                 /* Priority 3 */
```


CSS Syntax



CSS Selectors

1. Simple Selectors

name, #id, .class

```
p { }  
#main { }  
.note { }
```

2. Combinators

select by relationship

```
div p      (descendant)  
div > p    (child)  
p + h1     (adjacent)  
p ~ ul     (sibling)
```

3. Pseudo-class

state using :colon

```
a:hover  
a:visited  
input:focus  
li:first-child
```

4. Attribute Selectors

select by attribute [brackets]

```
[href]  
[type="text"]  
[class~="note"]  
[lang|"en"]
```

5. Pseudo-element Selectors

part of element using ::double-colon

```
p::first-line  
p::first-letter  
h1::before  
h2::after
```

The CSS element Selector

Element Selector

Selects ALL HTML elements with the given tag name on the page.

```
p {  
  text-align: center;  
  color: red;  
}
```

The CSS id Selector

ID Selector #name

Uses the id attribute. Each id must be unique on the page — selects exactly one element.

Syntax

Write a hash (#) followed by the element's id value to target it in your CSS.

```
#para1 {  
  text-align: center;  
  color: red;  
}
```

The CSS id Selector

⚠ **Note on Numeric IDs**

HTML accepts numeric id values — but CSS selectors cannot start with a digit, two hyphens, or hyphen+digit.

```
#para1 {  
  text-align: center;  
  color: red;  
}  
  
/* Check demo: css_1.html */
```

The CSS class Selector

Class Selector `.classname`

Selects all HTML elements that have a specific class attribute value.

Syntax

Write a period (.) followed by the class name to target elements with that class.

```
.center {  
    text-align: center;  
    color: red;  
}  
  
/* Check demo: css_2.html */
```

CSS Class Selector

Element-Specific Class

Use element.class notation to apply a class only to elements of a specific tag type.

```
p.center {  
  text-align: center;  
  color: red;  
}  
  
'<p class="center">Only this paragraph is styled.</p>'  
  
/* Check demo: css_3.html */
```

CSS Class Selector

Multiple Classes on One Element

An HTML element can reference more than one class at a time — separate class names with spaces.

```
<p class="center large">This paragraph refers to two classes.</p>  
  
/* .center applies text-align:center; .large applies font-size:2em */  
  
/* Check demo: css_4.html */
```


The CSS Universal Selector

Universal Selector *

The asterisk (*) selects ALL HTML elements on the page — useful for global resets.

```
* {  
  text-align: center;  
  color: blue;  
}  
  
/* Check demo: css_5.html */
```

CSS Selectors

1. Simple Selectors

name, #id, .class

```
p { }  
#main { }  
.note { }
```

2. Combinators

select by relationship

```
div p      (descendant)  
div > p    (child)  
p + h1     (adjacent)  
p ~ ul     (sibling)
```

3. Pseudo-class

state using :colon

```
a:hover  
a:visited  
input:focus  
li:first-child
```

4. Attribute Selectors

select by attribute [brackets]

```
[href]  
[type="text"]  
[class~="note"]  
[lang|"en"]
```

5. Pseudo-element Selectors

part of element using ::double-colon

```
p::first-line  
p::first-letter  
h1::before  
h2::after
```

The CSS Grouping Selector

Grouping Selector

When multiple selectors share the same declarations, group them with commas to avoid repetition and reduce code.

```
/* BEFORE — repetitive */
h1 { text-align:center; color:red; }
h2 { text-align:center; color:red; }
p  { text-align:center; color:red; }

/* AFTER — grouped */
h1, h2, p { text-align: center; color: red; }
```

The CSS Grouping Selector

Grouping Syntax

Separate each selector with a comma. Less code, easier to maintain.

```
h1, h2, p {  
  text-align: center;  
  color: red;  
}  
  
/* Check demo: css_6.html */
```

Summary of All CSS Simple Selectors

Selector	Example	Example description
<u>.class</u>	.intro	Selects all elements with class="intro"
<u>#id</u>	#firstname	Selects the element with id="firstname"
<u>*</u>	*	Selects all elements
<u>element</u>	p	Selects all <p> elements
<u>element,element,..</u>	div, p	Selects all <div> elements and all <p> elements

CSS Combinator Selectors

Descendant (space)

All descendants at any depth.

Child (>)

Direct children only.

Adjacent Sibling (+)

Immediately following sibling.

General Sibling (~)

All following siblings.

Descendant Selector

Descendant Selector (space)

Matches ALL elements that are descendants of a specified element, at any nesting depth.

```
div p {  
    background-color: yellow;  
}  
  
/* Selects ALL <p> elements inside ANY <div> */  
/* including nested ones */  
/* Check demo: css_7.html */
```

Child Selector

Child Selector >

Selects only DIRECT children — not grandchildren or deeper nested elements.

```
div > p {  
  background-color: yellow;  
}  
  
/* Only <p> that are direct children of <div> */  
/* Try adding > in css_7.html to see the difference */
```


Adjacent Sibling Selector

Adjacent Sibling Selector +

Selects the element immediately following the specified element. Siblings must share the same parent.

```
div + p {  
    background-color: yellow;  
}  
  
/* Only <p> placed directly AFTER <div> */  
/* Try adding + in css_7.html */
```

General Sibling Selector

General Sibling Selector ~

Selects ALL following siblings of a specified element that share the same parent.

```
div ~ p {  
    background-color: yellow;  
}  
  
/* All <p> siblings AFTER <div> (not before) */  
/* Try adding ~ in css_7.html */
```

All CSS Combinator Selectors

Selector	Example	Example description
<u><i>element element</i></u>	div p	Selects all <p> elements inside <div> elements
<u><i>element>element</i></u>	div > p	Selects all <p> elements where the parent is a <div> element
<u><i>element+element</i></u>	div + p	Selects all <p> elements that are placed immediately after <div> elements
<u><i>element1~element2</i></u>	p ~ ul	Selects every element that are preceded by a <p> element

CSS Pseudo-classes

What is a Pseudo-class?

Defines a special state of an element — not selectable by normal selectors.

Syntax

selector:pseudo-class { property: value; }

```
a:hover { color: hotpink; }           /* hover state */  
input:focus { border-color: blue; } /* focused input */  
li:first-child { font-weight: bold; } /* first in list */
```

Anchor Pseudo-classes

Anchor Pseudo-classes

Links can be styled differently based on their current state: unvisited, visited, hovered, or active.

```
a:link      { color: #FF0000; } /* unvisited link */
a:visited   { color: #00FF00; } /* visited link   */
a:hover     { color: #FF00FF; } /* mouse over    */
a:active    { color: #0000FF; } /* selected link */

/* Check demo: css_8.html */
```

Pseudo-classes and CSS Classes

Combining Pseudo-class + CSS Class

Narrow down pseudo-class styling to only elements with a specific class — fine-grained control.

```
a.highlight:hover {  
    color: #ff0000;  
}  
  
/* Only <a class='highlight'> turns red on hover */  
/* Normal <a> links are unaffected */
```

Hover on <div>

Hover on div

:hover works on any element — not just links. Applied to a <div> to change background on mouse-over.

```
div:hover {  
  background-color: blue;  
}  
  
/* Check demo: css_9.html */
```

Simple Tooltip Hover

Simple Tooltip Pattern

Hide a `<p>` by default. Show it only when hovering over its parent `<div>` — a pure CSS tooltip.

```
p {  
  display: none;  
  background-color: yellow;  
  padding: 20px;  
}  
  
div:hover p {  
  display: block;  
}
```

/ Check demo: css_10.html */*

All CSS Pseudo Classes

:link

Unvisited links

:visited

Visited links

:hover

Mouse over element

:active

Currently clicked

:focus

Input has focus

:checked

Checked checkbox/radio

:disabled

Disabled form element

:first-child

First child of parent

:last-child

Last child of parent

:nth-child(n)

nth child element

:not(selector)

NOT matching selector

:root

Document root

CSS Pseudo-elements

CSS Pseudo-elements

Style a specific PART of an element — first letter, first line, or injected content.

Syntax **::double-colon**

selector::pseudo-element { property: value; } — Note: :: (two colons) distinguishes from pseudo-classes

```
p::first-letter { font-size: 200%; color: #8A2BE2; }  
p::first-line   { color: #0000FF; font-variant: small-caps; }  
h1::before { content: url(icon.gif); }  
h1::after  { content: url(icon.gif); }
```

The ::first-line Pseudo-element

::first-line Pseudo-element

Applies styles to the FIRST LINE ONLY of a text block — the browser determines what is the first line based on viewport width.

```
p::first-line {  
  color: #ff0000;  
  font-variant: small-caps;  
}  
  
/* Formats only the first rendered line of <p> text */  
/* Check demo: css_11.html */
```

All CSS Pseudo Elements

::first-line

Styles first rendered line of element

::first-letter

Styles first letter — like a dropcap

::before

Inserts content before element content

::after

Inserts content after element content

::marker

Styles list item markers

::selection

Styles selected/highlighted text

CSS Attribute Selectors

[attribute] Selector

Selects all elements that have the specified attribute — regardless of its value.

[attribute=value] Selector

Selects elements where the attribute equals exactly the specified value.

```
a[target]           { background-color: yellow; }  
a[target='_blank']  { background-color: green; }  
input[type='text']  { border: 2px solid blue;   }  
input[type='submit']{ background-color: red;     }
```

CSS [attribute="value"] Selector

[attribute=value] Selector

Exact match — selects elements where the attribute equals exactly the specified value.

```
a[target='_blank'] {  
  background-color: yellow;  
}  
  
/* Selects all <a> with target='_blank' */  
/* (opens link in new tab) */
```

All CSS Attribute Selectors

[attr]

Has the attribute

[attr=v]

Equals exactly value

[attr~=v]

Word in space-sep list

[attr|=v]

Equals or starts lang-

[attr^=v]

Starts with value

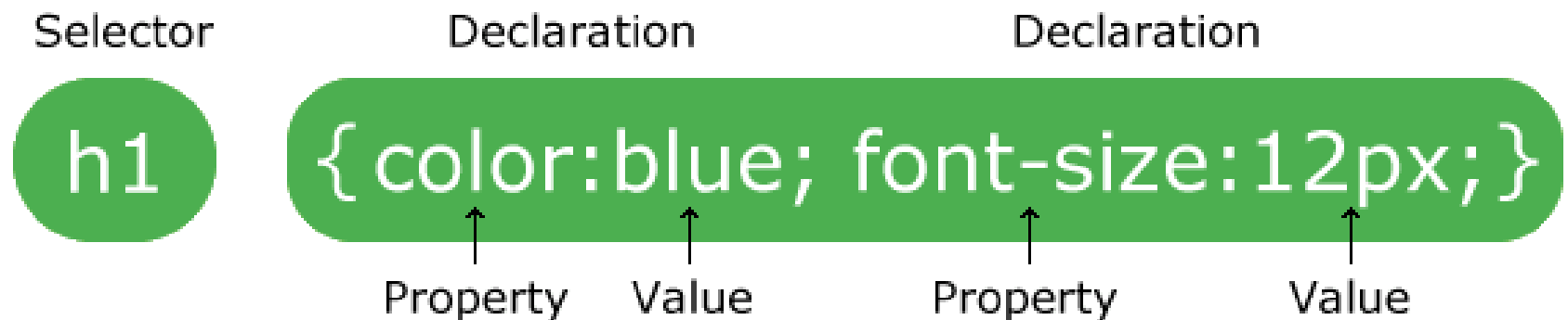
[attr\$=v]

Ends with value

[attr*=v]

Contains value anywhere

CSS Syntax



CSS Comments

CSS Comments

Comments are completely ignored by browsers. Use them to explain your code or temporarily disable rules.

```
/* Single-line comment */  
p { color: red; }  
  
/* This is  
   a multi-line  
   comment */  
p { color: red; }  
  
/* p { color: blue; }  <- temporarily disabled */
```

CSS Colors

Named Color

`color: Tomato;`
→ renders as: Tomato

HEX Color

`color: #FF6347;`
→ renders as: #FF6347

RGB Color

`color: rgb(255,99,71);`
→ renders as: rgb(255,99,71)

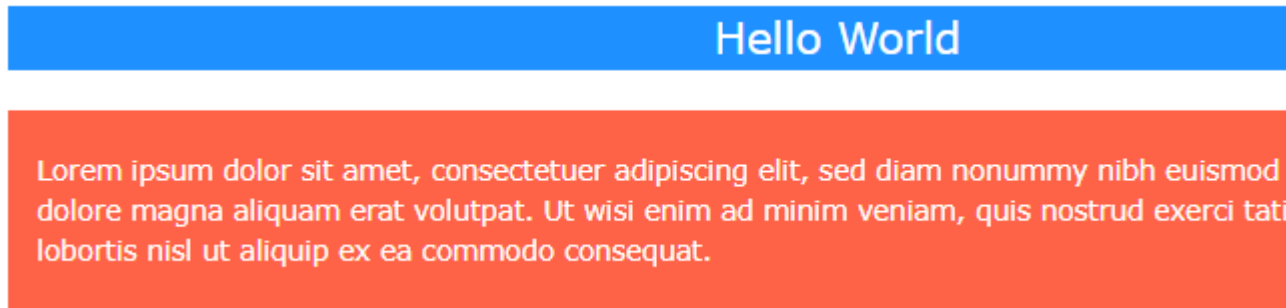
HSL Color

`color: hsl(9,100%,64%);`
→ renders as: hsl(9,100%,64%)

RGBA and HSLA add opacity: `rgba(255,99,71, 0.5)` | `hsla(9,100%,64%, 0.5)`

CSS Background Color

- ▶ You can set the background color for HTML elements:



- ▶ `<h1 style="background-color:DodgerBlue;">Hello World</h1>`
`<p style="background-color:Tomato;">Lorem ipsum...</p>`

CSS Text Color

CSS Text Color

Use the color property to set the foreground text color of any element — works with all color formats.

```
<h1 style="color:Tomato;">Hello World</h1>
<p style="color:DodgerBlue;">Lorem ipsum...</p>
<p style="color:MediumSeaGreen;">Ut wisi enim...</p>

/* In a stylesheet: */
h1 { color: #1A3575; }
p { color: rgb(0,100,0); }
```

Hello World

Lorem ipsum dolor sit amet, consectetur adipiscing
magna aliquam erat volutpat.

Ut wisi enim ad minim veniam, quis nostrud
consequat.

CSS Border Color

Set border color using color names, hex values, RGB, or HSL

Tomato Border

```
h1 { border: 2px solid  
  Tomato; }  
#ff6347 hex value  
Warm red tone
```

DodgerBlue Border

```
h1 { border: 2px solid  
  DodgerBlue; }  
#1e90ff hex value  
Bright blue tone
```

Violet Border

```
h1 { border: 2px solid  
  Violet; }  
#ee82ee hex value  
Light purple tone
```

```
/* CSS Border Color Examples */  
h1 { border: 2px solid Tomato; }  
h2 { border: 2px solid DodgerBlue; }  
h3 { border: 2px solid Violet; }  
  
/* Using hex, rgb, or hsl values */  
div { border: 3px solid #ff6347; }  
p { border: 3px solid rgb(30, 144, 255); }  
span { border: 3px solid hsl(300, 76%, 72%); }
```

CSS RGB Colors

rgb(red, green, blue) — each value from 0 to 255

Red Channel

rgb(255, 0, 0) = Red
Red at max (255)
Green & Blue = 0
Result: Pure Red

Black & White

rgb(0, 0, 0) = Black
rgb(255,255,255) = White
All channels equal
= shade of gray

Mixed Colors

rgb(255,165,0) = Orange
rgb(0,128,0) = Green
rgb(75,0,130) = Indigo
Mix any shade!

```
/* CSS RGB Color Formula */  
color: rgb(red, green, blue);  
  
/* Examples */  
color: rgb(255, 0, 0); /* Red */  
color: rgb( 0, 128, 0); /* Green */  
color: rgb( 0, 0, 255); /* Blue */  
color: rgb( 0, 0, 0); /* Black */  
color: rgb(255, 255, 255); /* White */
```

CSS HEX Colors

#rrggbb — hex values from 00 to ff for each channel

Red in Hex

#ff0000 = Red
ff = max red
00 = no green
00 = no blue

Hex Shorthand

#fff = white
#000 = black
#f00 = red
Works when pairs match

Common Colors

#ff6347 = Tomato
#1e90ff = DodgerBlue
#ee82ee = Violet
#ffa500 = Orange

```
/* CSS HEX Color Format: #rrggbb */
color: #ff0000; /* Red */
color: #00ff00; /* Lime */
color: #0000ff; /* Blue */
color: #000000; /* Black */
color: #ffffff; /* White */

/* Shorthand (when pairs match) */
color: #f00; /* same as #ff0000 */
color: #fff; /* same as #ffffff */
```

CSS HSL Colors

hsl(hue, saturation%, lightness%) — intuitive color model

Hue (0-360°)

0 = Red
120 = Green
240 = Blue
Degrees on color wheel

Saturation %

0% = gray
50% = muted
100% = full color
Controls color shade

Lightness %

0% = black
50% = normal
100% = white
Controls brightness

```
/* CSS HSL Color Formula */
color: hsl(hue, saturation%, lightness%);

/* Examples */
color: hsl( 0, 100%, 50%); /* Red */
color: hsl(120, 100%, 50%); /* Lime */
color: hsl(240, 100%, 50%); /* Blue */
color: hsl( 39, 100%, 50%); /* Orange */
color: hsl( 0, 0%, 50%); /* Gray */
```


CSS Backgrounds

CSS background properties define visual effects behind elements

background-color

Sets a solid fill color

```
body { background-color:
  lightblue; }
```

Most basic property

background-image

Sets an image as bg

```
body { background-image:
  url('bg.jpg'); }
```

Repeats by default

More Properties

background-repeat

background-position

background-attachment

background (shorthand)

```
/* CSS Background Properties */
```

```
body {
  background-color:    lightblue;
  background-image:    url('bg.jpg');
  background-repeat:    no-repeat;
  background-position: center top;
  background-attachment: fixed;
}
```

```
/* Shorthand */
```

```
body { background: lightblue url('bg.jpg') no-repeat center; }
```

CSS Borders

Specify border style, width, and color for any element

Border Style

dotted dashed
solid double
groove ridge
none / hidden

Border Width

border-width: 5px;
border-top: thin;
border-right: medium;
border-bottom: thick;

Border Color

border-color: red;
border-color: #ff0000;
rgb(255,0,0)
Shorthand per side

```
/* CSS Border Properties */
p { border-style: solid; border-width: 3px; border-color: tomato; }

/* Shorthand: width style color */
p { border: 3px solid tomato; }

/* Per-side control */
p { border-top: 2px dotted blue; }
p { border-right: 2px solid red; }
p { border-bottom: 2px dashed green; }
p { border-left: 2px double violet; }
```

CSS Margins

Space outside an element's border — full control per side

Per-Side Margins

margin-top
margin-right
margin-bottom
margin-left

Shorthand Values

1 value: all sides
2 values: top/btm, lr
3 values: top, sides, btm
4 values: T R B L

Auto & Centering

margin: auto;
Centers block elements
margin: 0 auto;
Top/bottom=0, centered

```
/* CSS Margin - Per Side */
p { margin-top: 100px; margin-bottom: 100px;
    margin-right: 150px; margin-left: 80px; }

/* Shorthand: top right bottom left */
p { margin: 100px 150px 100px 80px; }

/* Center a block element horizontally */
div { width: 300px; margin: 0 auto; }
```

CSS Padding

Space inside an element's border — between content and the border

Per-Side Padding

padding-top
padding-right
padding-bottom
padding-left

Margin vs Padding

Margin = outside border
Padding = inside border
Both add space
Different layout impact

Box Sizing Tip

box-sizing: border-box;
Padding included in width
No unexpected overflow
Use on all elements!

```
/* CSS Padding - Per Side */
div { padding-top: 50px; padding-right: 30px;
      padding-bottom: 50px; padding-left: 80px; }

/* Shorthand: top right bottom left */
div { padding: 50px 30px 50px 80px; }

/* Best practice - include in all stylesheets */
* { box-sizing: border-box; }
```

CSS Responsive

Adapt layouts for desktops, tablets, and phones using media queries

The Problem

Same content shown on:
4K desktops, tablets
5-inch phones
Must look good
everywhere

Media Queries

@media (max-width:
768px)
Apply rules only when
screen matches condition
Different layout per device

Mobile-First

Design for mobile first
Then scale up for desktop
Progressive Enhancement
See Demo: [css_13.html](#)

```
/* Mobile-First Responsive Layout */  
/* Base styles for mobile */  
.container { width: 100%; padding: 10px; }  
  
/* Tablet and above */  
@media (min-width: 768px) {  
  .container { max-width: 720px; margin: 0 auto; }  
}  
  
/* Desktop and above */  
@media (min-width: 1024px) {  
  .container { max-width: 960px; }  
}
```

CSS Grid

Two-dimensional layout system with rows and columns — no floats needed

Grid Container

display: grid;
Defines grid context
All direct children
become grid items

Defining Columns

grid-template-columns:
 repeat(3, 1fr);
fr = fractional unit
Equal width columns

Grid Gap

gap: 10px;
Space between cells
row-gap, column-gap
Replaces grid-gap

```
/* CSS Grid Layout */
.grid-container {
  display: grid;
  grid-template-columns: repeat(3, 1fr);
  gap: 10px;
}
.grid-item {
  background-color: rgba(255,255,255,0.8);
  padding: 20px;
  text-align: center;
}
```

CSS Grid Example

You can **combine** the `fr` unit with **any other CSS units** as well. For instance, in the example below, I used the `60% 1fr 2fr` ratio for my grid.



Other CSS

Many more CSS topics — explore further at [w3schools.com/css](https://www.w3schools.com/css)

Height/Width

Box Model

Outline

CSS Text

CSS Fonts

CSS Icons

Also: CSS Tables, Links, Lists, Position, Max-width — visit [w3schools.com/css](https://www.w3schools.com/css)

CSS Frameworks

Pre-built CSS systems — Bootstrap and Tailwind dominate the market today

Bootstrap

Component-based classes
Pre-styled components ready
Great for rapid prototyping
Largest ecosystem

Tailwind CSS

Utility-first approach
Tiny custom classes
Highly customizable
Fastest growing (2025)

UIKit

Bulma

**Bootstrap vs Tailwind —
which is better?**

Modern CSS You Should Know

Variables, Flexbox, Grid, and Transitions — essential modern CSS

CSS Variables

```
:root { --primary:
#007BC4;
  --gap: 1rem; }
p { color: var(--
primary);
  margin: var(--gap); }
```

CSS Flexbox

```
.container {
  display: flex;
  justify-content:
    space-between; }
```

CSS Transitions

```
button {
  transition:
    background
      -color 0.3s ease; }
button:hover {
  background:
    navy; }
```

```
/* CSS Grid: two-dimensional layout */
.grid {
  display: grid;
  grid-template-columns: repeat(3, 1fr);
  gap: 1rem;
}

/* Use Grid for 2D, Flexbox for 1D layouts */
/* Grid also covered in detail on Slide 62 */
```

CSS Custom Properties (Variables)

Define once in :root — change one variable, update the whole site

Theming

Swap one :root block to switch light/dark mode
No more searching 50 files for a hex color

Responsive Scaling

Change --font-size-base at a breakpoint and everything scales automatically

Maintainability

Prefer variables over Sass/Less for new projects
Supported in all modern browsers today

```
:root {  
  --brand-color:    #00AD72;  
  --font-size-base: 16px;  
  --spacing-md:     1.5rem;  
}  
h1 { color: var(--brand-color); }  
p  { margin-bottom: var(--spacing-md); }  
  
/* Dark mode override */  
@media (prefers-color-scheme: dark) {  
  :root { --brand-color: #00FF99; }  
}
```

CSS Specificity - Why Your Styles Get Overridden

Higher specificity score wins when two CSS rules conflict

Low — 0-0-1

Element selector
`p { color: red; }`
`div, h1, span...`
Score: 0-0-1

Medium — 0-1-0

Class selector
`.card { color: red; }`
Attribute `[type=text]`
Score: 0-1-0

High — 1-0-0

`#header { color: red; }`
ID selector
Avoid IDs for styling!
Use classes instead

```
/* Specificity conflict example */
p { color: red; } /* 0-0-1 LOSES */
#content p { color: blue; } /* 1-0-1 WINS */

/* Inline style always wins except !important */

/* Best practice */
/* Use .card instead of #card for styling */
/* Avoid nesting more than 3 levels deep */
/* Use !important only as last resort */

/* Inline -> ID > Class -> Element */
```

AI in CSS Development

Generate, fix, and refactor CSS faster using AI tools

Generate CSS

'Write CSS for a 3-col responsive card grid using Flexbox'
AI writes the boilerplate

Fix Broken Styles

Paste CSS + describe bug
AI diagnoses specificity conflicts + suggests fix
Faster than Stack Overflow

AI CSS Tools

GitHub Copilot: IDE assist
Claude/ChatGPT: explain
Figma AI: design to CSS
Cursor: full refactoring

```
/* Good AI Prompting Pattern for CSS */  
/* Describe element + visual + constraints */
```

```
"Write CSS for sticky navbar, mobile-first"  
"Generate :root block with dark-mode palette"  
"Write keyframe: fade-in and slide-up"  
"Convert px layout to rem and clamp()"
```

AI in CSS Testing and Quality

AI-powered tools for visual regression, accessibility, and cross-browser QA

Visual Regression

Percy / Chromatic:
screenshots before + after
AI flags pixel differences
Playwright + AI tests

Accessibility Audit

axe DevTools AI
Lighthouse AI (Chrome)
WCAG contrast checks
Prompt: check #007BC4

CSS Linting + QA

Stylelint + AI
Auto-detect duplicates
Unused selectors
Ordering issues

```
/* AI Prompt examples for CSS QA */  
  
/* Accessibility */  
"Check #007BC4 with white text: WCAG AA?"  
  
/* Cross-browser QA */  
"Suggest breakpoints for mobile, tablet, desktop"  
  
/* Linting */  
"Review CSS for redundancy and specificity issues"  
  
/* BrowserStack AI: tests across 30+ browsers */
```

WCAG = Web Content Accessibility Guidelines

Next

JavaScript